

ALGORITHMS

* Rate of Growth: (O/w)

$$\frac{1}{n} < K < \log n < \sqrt{n} < n < n^2 < 2^n$$

$$< 3^n < n! < n^n$$

Small
O/w

$$B(n) \leq A(n) \leq W(n)$$

* Big-Oh (O):

$$f(n) = O(g(n))$$

$$f(n) \leq c * g(n)$$

$$c > 0$$

$$n_0 > 0$$

$$n \geq n_0$$

* Big Omega (Ω):

$$f(n) = \Omega(g(n))$$

$$f(n) \geq c * g(n)$$

$$c > 0$$

$$n_0 \geq 0$$

$$n > n_0$$

* Theta (Θ):

$$f(n) = \Theta(g(n)) \text{ if } f(n) = O(g(n)) \text{ \& } f(n) = \Omega(g(n))$$

Tight-bound

$$8n^2 / 5n^2 \Rightarrow f(n) = O(g(n))$$

$$f(n) = g(n) \Rightarrow g(n) = O(f(n))$$

Note:

$$O(n^4) \rightarrow O(n^4, n^5, \dots)$$

$$O(n^4) \rightarrow \Omega(n^4, n^3, \dots)$$

$\rightarrow O, \Omega$ may or may not tight.

$\rightarrow O, \omega$ always loose bound.

$$O; f < g$$

$$\omega; f > g$$

$$d(n) = O(f(n)) \Rightarrow a d(n) = O(f(n))$$

$$d(n) + e(n) = O(f(n) + g(n)) / \max(f, g)$$

$$d(n) \cdot e(n) = O(f(n) \cdot g(n))$$

$$f(n) \text{ polynomial}; f(n) = O(n^d)$$

$$n^x = O(a^n)$$

$$\log n^x = O(\log n)$$

$$\log n^x = O(n^y) \quad y > 0$$

$$x > 0$$

$$O(f(n) + g(n)) = O(\max(f, g))$$

$$\Omega(f(n) + g(n)) = \Omega(\max(f, g))$$

$$O(f(n)) + O(g(n)) = O(\max(f, g))$$

$$O(f(n)) * O(g(n)) = O(f(n) * g(n))$$

$\rightarrow$ Asymptotic Notation does not follow toichotomy prop^s.
may/may not

$$\rightarrow \text{Ref}^*: O, O, \Omega$$

$$\rightarrow \text{trans}^*: \text{ALL}$$

$$\rightarrow \text{Symm}: \text{only } O$$

$$\rightarrow \text{trans}^* \text{ Symm}: (\text{ALL} - O)$$

$$\bullet \text{ If Small } O \Rightarrow \text{ Big Oh } (\leq)$$

$$\bullet \text{ If } \omega \Rightarrow \Omega \text{ (may/may not)}$$

* Time-Complexity of Recu^s Al^s

$$1. T(n) = T(n-1) + C \Rightarrow \begin{matrix} n = c \cdot n \\ n = c \cdot n^2 \\ n = c \end{matrix}$$

$$2. T(n) = T(\sqrt{n}) + C \Rightarrow O(\log \log n)$$

$$3. T(n) = 2 T(n-1) + C \Rightarrow O(2^n)$$

$$4. T(n) = 2 T(\sqrt{n}) + C \Rightarrow O(\log n)$$

$$* a \log b^x = x \log b^a$$

$$5. T(n) = 2 T(n/2) + C \Rightarrow O(n)$$

$$6. T(n) = 2 T(n/2) + n \Rightarrow O(n \log n)$$

$$7. T(n) = n T(\sqrt{n}) + n \Rightarrow O(n \log \log n)$$

$$8. T(n) = T(\sqrt{n}) + C \Rightarrow O(\log \log n)$$

$$9. T(n) = T(n/2) + C \Rightarrow O(\log n)$$

$$10. T(n) = 2 T(n/2) + n \log n \Rightarrow O(n \log^2 n)$$

* Loop

$\Rightarrow$ if unconditional break; $O(1)$ of that loop.

$$\Rightarrow \text{while } (c=2) \{$$

$$\quad \quad \quad \}$$

$$\Rightarrow \text{Nested Indepd loop} = m * n$$

$$\Rightarrow \text{"dipped" loop} = \text{analysis.}$$

$$\Rightarrow \text{consecv loop} = O(\max(m, n))$$

$$\bullet \text{ for } (i=1; i \leq n; i=i+b) \Rightarrow O(n)$$

$$\bullet \text{ " " " } i=i*b \Rightarrow O(\log n)$$

$$\bullet \text{ while } (a < n) \{$$

$$\quad b = b+1 \Rightarrow O(\sqrt{n})$$

$$\quad a = a+b \Rightarrow O(\sqrt{n})$$

* Space-Complexity

$\Rightarrow$ additional space except hpt.

$\Rightarrow$ In Recursion: Recursion Stack is the auxiliary space. $O(n)$

$$\text{So } SC = SC$$

* Recursive Tree:

$$1. T(n) = 2 T(n/2) + n$$

Cost of recursion at that level.

$$n/2 \quad n/2 \quad n$$

$$x = \log n \quad n \log n$$

$$2. T(n) = T(n/3) + T(2n/3) + n$$

$$\Omega(\log_3 n \cdot n) \& O(\log_{3/2} n \cdot n)$$

$$\Theta = n \cdot \log n$$

log₃ depth.

$$* T(n) = T(n-1) + T(n-2) + 1$$

$$\begin{matrix} n & \rightarrow & 1 (2^0) \\ n-1 & \rightarrow & 2 \\ n-2 & \rightarrow & 4 \\ n-3 & \rightarrow & 8 \\ n-4 & \rightarrow & 16 \end{matrix}$$

$$K \Rightarrow 2^{K-1}$$

Left most: n level

Right most: $n/2$ level.

Total cost till K : $2^0 + 2^1 + 2^2 + \dots + 2^{K-1} = 2^K - 1$

$$\therefore L.m = O(2^n) \rightarrow \max$$

$$\therefore R.m = O(2^{n/2}) \rightarrow \min.$$

* Note:

$$2^n \geq n^n$$

* Sorting:

$\rightarrow$ Inplace: atmost S.C $O(1)$ + $(\log n)$ Recursion Stack.

$\Rightarrow$ Stable: relative position are same.

$\Rightarrow$ Inversion: pairs of indices (i, j) $(i < j)$ s.t. $A[i] > A[j]$

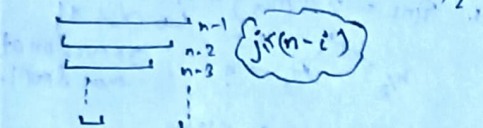
$$0 \leq (inv) \leq \frac{n(n-1)}{2}$$

Bubble Sort

in every i^{th} pass, i^{th} max element will place at its correct posn.

B.C $(n-1)^2$
W.C $(n-1)^2$

No. of Comp^s $(n-1)^2$
No. of Swaps $\frac{n(n-1)}{2}$



Algo: 02

B.C $(n-1)$
W.C $\frac{n(n-1)}{2}$

S.C = $O(1)$ Inplace Stable.

Selection Sort

find 1st min in every i^{th} pass & swap.

B.C $O(n^2)$
W.C $O(n^2)$

No. of Comp^s $O(n^2)$
No. of Swaps $(n-1)$

Inplace - unstable

always take $(n-1)$ passes
Every pass, exactly one swap
take least swap in W.C = $O(n^2)$

Insertion Sort

$(n-1)$ passes.

B.C $(n-1)$
W.C $\frac{n(n-1)}{2}$

No. of Comp^s $O(n^2)$
No. of Swaps $\frac{n(n-1)}{2}$

Inplace - stable

if input is presorted then it takes $O(n)$; d = no. of inv.

Radix Sort

(Non-comparison)

No. of passes = No. of digits in the max elements.

L to R } Bottom to top.

T.C = $O(d * (n+b)) \approx d^2 n$

base of input = b $(d = \log_b n)$

n : no. of element

d : no. of digit in max elem

Not Inplace - Stable

Note

expected no. of inversion in randomly $(1..n)$ permutation.

$$\frac{n(n-1)}{2} * \frac{1}{2}$$

Insertion sort use Right Shifting

Divide & Conquer

- Divide: mandatory
- combine: optional.

min-max Algo:

1st - 1st for min: $2(n-1)$
for max: $2(n-1)$

1st - 1st " min: $(n-1)$
" max: $2(n-1)$

B.C = $(n-1)$ avg. $\frac{3(n-1)}{2}$
W.C = $2(n-1)$

Divide & C T.C = $2 * T(n/2) + 2$

$$T.C. = \frac{3n-2}{2}$$

for all B.C, W.C, Avg.C.

S.C $\rightarrow O(1)$: Non D.C.

$O(\log n)$ Recursion Stack.

Binary Search

assuming start & end/height = 1

$$\log_2(n+1) \leq n \leq n$$

if every el has exactly i node.

$$n = 1 + 2 + 3 + \dots + h$$

$$mid = \left\lfloor \frac{l+h}{2} \right\rfloor$$

$$T(n) = T(n/2) + C ; n \geq 1$$

$$O(\log_2 n)$$

only divide. no combine.

Merge Sort

Stable, Inplace

Merging $\rightarrow B.C = \min(m, n)$

W.C = $(m+n-1)$ (comparisons)

$$mid = \left\lfloor \frac{l+h}{2} \right\rfloor$$

single element.

merging: $O(n)$

$$T(n) = 2T(n/2) + b.n \rightarrow O(n \log n)$$

$$S.C = (n + \log n)$$

Note

2-way merging applicable

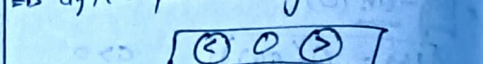
Quick Sort

partitioning: $O(n)$

pivot at start & end after pass will cause $w.c O(n^2)$

other pivot will cause $O(n \log n)$

after partitioning



B.C/A.C $\frac{n(n-1)}{2}$ W.C

$$T(n) = 2T(n/2) + n$$

$$O(n \log n)$$

Recursion Stack: $O(n)$

inplace - unstable

Note

50 sec $\rightarrow n \log n$ (this is not linear)

$$50 \text{ sec} = C * n \log n$$

calculate C

Master Methods

$$T(n) = aT(n/b) + F(n) ; \begin{cases} a \geq 1 \\ b > 1 \end{cases}$$

$$1: f(n) < n \log_b a ; O(n \log_b a)$$

$$2: f(n) > n \log_b a \text{ \&\& } a f(n/b) \leq c.f(n) ; O(f(n))$$

$$3: F(n) = (n \log_b a * (\log n)^k)$$

$$k \geq 0 ; O(n \log_b a * (\log n)^{k+1})$$

$$k \leq -1 ; O(n \log_b a * \log \log n)$$

Note:

for n elements: $2(n-1)$ merge sort will call.

maintain array: $O(\log n)$ for the search

2 way Merging: $n \log n$

- * Matrix Multiplication;
for Sq matrix.
- Matrix Addition: $O(n^2)$ _{sub} $n \times n$
 - Matrix MultiP: $O(n^3)$ $n \times n$
4 addition
8 multiplc

using DnC: $T(n) = 8T(n/2) + O(n^2)$
 $\rightarrow O(n^3)$ Δ $T(n)$ 2×2 matrix

using Strassen's MM:
 $T(n) = 7T(n/2) + bn^2$
 $\rightarrow O(n^{2.81})$ $n \gg$

- Space Complexity:
- Non DnC: $O(1)$
- Simple DnC: Recursⁿ Stack $O(\log_2 n)$
- Strassen: n^2 (P, Q, R, S, T, U, V)

* Long-Int Multiplication;
2 way split.

K-way split: DnC, $T(n) = K^2 T(n/K) + bn$
AK optim²: $T(n) = (K^2 - 1)T(n/K) + bn$
gcd optim: $T(n) = (2K - 1)T(n/K) + bn$

* Backtracking;
① N-Queens: $\rightarrow n!$ size of solution space.
to get 1st feasible solⁿ $\rightarrow$ use Dfs, to search
16 state required $\rightarrow$ apply bound funcⁿ to stop that path.

② Sum of Subset:
• target (m), array (-----n)
• No. of subset: 2^n
• variable sized tuple as feasible solⁿ $n=4$ $2^4 = 16$ state space.

③ Graph coloring: min color to that g.

④ Branch-Bounding: BFS + Bound funcⁿ (prune)

* Note:
Backtracking Branch Bound
- N-Queens - N-Queens
- Graph coloring - Travelling Salesman
- Sol - Job Seqⁿ
- 0/1 Knapsack - 0/1 Knapsack
- Hamiltonian Cycle - 15 puzzle prob

* GREEDY ALGORITHM:
- Local optimization.
- Fractional Knapsack $\checkmark$
- merging, Job Sequence.

o Fractional Knapsack:
- $\left\lfloor \frac{p}{w} \right\rfloor$: dec^r order (Profitize)
- if $w_i > M_x$, skip that item and move further.

Note:
N-Queens: Size of solⁿ space: $n!$
optimal solⁿ $\rightarrow$ value $\rightarrow$ unique.

o Job Sequence with Deadline:
• no. of Job = max (Deadline)
• profitize with Dec (Profit) order
• arrange rank wise with max-possible deadline. (can be shift left)
 n^2

o Optimal merging patterns:
• Select 2 smallest file & merge (m+n), also consider this new file on next iteration.
• Recordⁿ move: m+n
• Comparisons: (m+n-1) : we min(m,n): BC

• Total R.M: $\sum_{i=1}^n d_i q_i$ size of file
• Implementation (distance from root)
LL: $O(n^2)$ $\rightarrow$ heap $(n \log n)$

o Huffman Encoding:
- for N elements: $\lceil \log_2(N) \rceil$ bits required.
- possible only with non-uniform encoding.
- no. of bits to encode $\propto \frac{1}{\text{freq of char}}$

- unique encoding.
(Proof)
- Avg bits/char = $\sum \text{edge} \left(\frac{\text{sum of } \downarrow}{\text{nodes}} \right)$
- assigning bit = left branch - 0
right " - 1

(Proof)
- Avg bit/char = $\frac{\text{Total Encoded L}}{\text{Total no. of char. in string. (Not unique)}}$
 $(\text{freq} \times \text{code length})$

o MCS:
 $\rightarrow$ Acyclic + (n-1) edges.
• Adjⁿ matⁿ: Adj list $O(n^2)$: Adj list $O(n+e)$
(Dense G) : $O(n + \frac{n^2}{2})$ (sparse)²

• no. of edges must be removed to get spanning tree.
 $e = (n-1)$
• tree with n nodes have (n+1) null links

$\rightarrow$ Kruskal Algo:
• Sort Edge in incr, then connect until (n-1) edge & no cycle.
 $\rightarrow (n \log n)$
 $\rightarrow$ mcostsⁿ will be always same.

$\rightarrow$ Prim's Algo:
• Source node, then connect with minⁿ connected nodes
• Consider all nodes which are connected.

Note:
 $\rightarrow$ If an edges are distinct then MCSⁿ by both are same. otherwise may or may not...
• (MCS)_{cost} always unique.
• for a cycle max edge will rem.

- from m edges, if u have to choose n edges: ${}^m C_n$
- * If e is the heaviest edge of some cycle in G , then every MST of G excluded e .
- Prim's: non-Heap & heap
 $O(n^2)$ $O(n \log n)$

Dijkstra's SSSP Algo:

- Greedy Algo. $\rightarrow$ Matrix $\rightarrow$ S.S.
- Bellman food = DP.
- once a vertex is selected, it doesn't go further relaxed.

D. SSSP $\neq$ MCS

$O(n^2)$ } heap
non-heap } $O(n \log n)$

Note:

- MST of a G is always Unique.
- Shortest path is not unique.

* DYNAMIC PROGRAMMING:

- Global optm. $\rightarrow$ coin change
- memorization (Recursive) $\rightarrow$ multi stage G
- Bottom-up (Tabulation/Iterat) $\rightarrow$ 0/1 Knapsack.

Fibonacci Series:

- $Fib(n) = Fib(n-1) + Fib(n-2)$
- $Fib(1) = 1$; $Fib(0) = 0$
- $T(n) = T(n-1) + T(n-2) + 1, n > 1$
- with dp: $O(n)$ } $O(2^n)$
- overlapping Subproblem.
- S.C: $O(n)$ auxiliary arr.

Note:

- DP: Independent Subproblem.
- DP: Overlapping Subproblem.
- Dijkstra: Greedy
- Bellman food: DP

- Bellman Food:
 - can give soln for -ve edge also But not -ve, w. cycle.
 - Dijkstra: only +ve wt.
 - once the -ve $\rightarrow$ may/may not.
 - v. is selected no relaxatn further.

Bellman: $(n-1)$ relaxation/iter.
D.P: $O(n \cdot e)$

- for each edge we do Relax
- then check for -ve wt. cycle.

Floyd warshall (APSP):

$A(i, j) = C(i, j)$ (initialization)
 $A^k(i, j) = \min_{i \leq k \leq n} \{A^{k-1}(i, k) + A^{k-1}(k, j)\}$

T.C: $O(n^2 + n^3) = O(n^3)$
 S.C: $O(n^2)$

0/1 Knapsack:

$K(n, m) = 0$; $n=0$ OR $m=0$
 $K(n, m) = K(n-1, m)$, $w_n > m$
 $K(n, m) = \max \left(K(n-1, m), K(n-1, m-w_n) + p_n \right)$
 T.C: $O(n \cdot m)$
 S.C: $O((n+1) \cdot (m+1))$

Sum of Subset:

- True, if there exist a subset sum Equal to target sum Equals to M .
- Generally, (2^n) for all subsets.
- $S(n, m) = \begin{pmatrix} S(n-1, m) \\ \text{OR} \\ S(n-1, m-w_n) \end{pmatrix}$ $(n \leq m)$
- $S(n, m) = \begin{cases} \text{true} & n=0 \text{ or } m=0 \\ \text{false} & m > 0 \end{cases}$
- $S(n, m) = \text{true}$; $m=0$ & $n \geq 0$
- T.C: $O(m \cdot n)$; $m = \text{target}$
 S.C: $O(m \cdot n)$; $n = \text{size}$

LCS:

- for string of size n :
- no. of substring: $\frac{n(n+1)}{2}$
- Subsequence.
- Substring.

no. of subsequence:
 non-Empty: $(2^n - 1)$
 Empty: 1

Recurrence:

$L(i, j) = 0$; $i=0$ OR $j=0$

$L(i, j) = 1 + L(i-1, j-1)$; $x[i] = y[j]$

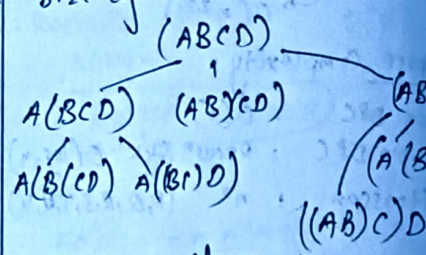
$L(i, j) = \max \left(L(i-1, j), L(i, j-1) \right)$; if not equal

T.C: $O(n \cdot m)$
 S.C: $O(n \cdot m)$
 T. distinct function call: $\Rightarrow (n+1)(m+1)$

mcm:

$A_{m \times n} B_{n \times p} = R_{m \times n \times p}$ Scalar mult

Size of $R = m \times p$



- Total no. of way to multiply n matrix chain. is

$Cat(n) = \frac{1}{(n+1)} 2^n$

$M[i, j] = \min \{ m[i, k] + m[k+1, j] + p_{i-1} \cdot p_k \cdot p_j \}$
 $i \leq k \leq j-1$

T.C: $O(n^3)$
 S.C: $O(n^2)$

Multi-stage G :

$C(i, j)$: cost of edge $i \rightarrow j$ vertex
 $Cost(i, j)$: cost of path from j vertex in stage i to reach destination.

$Cost(i, j) = \min (C(j, k) + Cost(i, k))$
 stage vertex

$k \in V_{i+1}$ at stage $(i+1)$

Base condn:

$Cost(i-1, j) = C(i, d)$

* Travelling Salesman Problem:

travelling exactly once.
cost minimize

$$C(1, K) + g(K, S - \{K\})$$

$$g(1, \{2, 3, 4\}) = \min[C(1, K) + g(K, S - \{K\})]$$

$$g(i, \emptyset) = C(i, v_0) \text{ Base Cond}$$

$$T.C = O(2^n \cdot n^2)$$

$$S.C = O(2^n \cdot \sqrt{n})$$

No solution in polynomial time.

* GRAPH ALGO:

DFS: LIFO → solve prob of SSSP
BFS: FIFO → can be LIFO using Queue.

Discovery time: time at which node got discovered first time.

Finishing time: at which node become dead (pop out).

Multiple DFS, BFS can be possible.

Tree Edge: present in the DFS.

Forward: Edge from A → B non-child desc.

Backward: Edge from Node to its ancestor

Cross: Edge neither from Ancestors nor from Descendant.

Multiple types of Edges can be possible but their union makes the Graph itself.

In any DFS, exactly 2 props hold

1. If interval $[d(u), f(u)]$ & $[d(v), f(v)]$ are entirely disjoint neither u nor v are descendant of the other.

2. Interval $(d(u), f(u))$ contains interval $(d(v), f(v))$ and u is a descendant of v .

3. Same for v is desc of u if u contains v .

* DAG:

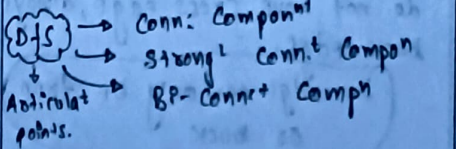
- Directed acyclic graph.
- Topological Sort.
- Sink = outdegree 0
- Source = in " 0

Note:

- In D/F times, it's only mandatory to have time in increasing order.
- 1 single node is also conn.
- overlapping intervals are always connected.
- DFS & BFS: S.C = $O(n)$
- T.C: adjacency Mat: n^2
list: $(n \cdot e)$

* Application of DFS & BFS:

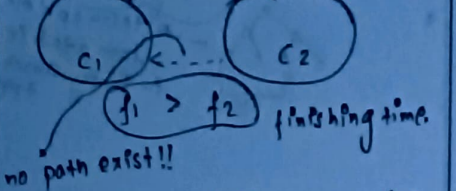
- Both DFS & BFS:
 - can be used to determine if u & v are connected or not / cycle detection.



Conn. Comp: Max subgraph that is connected.

Strongly Conn. Comp: Both path exist. $x \rightarrow y$ & $y \rightarrow x$

SCC are DAG & DFS joint.



To find no. of Bi-connected G.

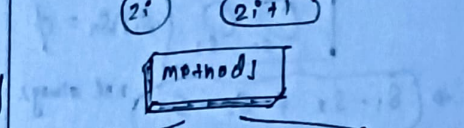
Double the A.P are separate. Separated graph is called Bi-connected G.

Because it does not have A.P.

If G is not Bi-conn then its maximal subgraph which is Bi-conn is a bi-connected G.

* Heap:

- Heap is a CBT.
- Max Heap: root $\geq$ LST / RST
- Min: root $\leq$ LST / RST



methods

- Insertion: $O(n \log n) / \Omega(n)$
- Heapify / Build Heap: $O(n)$
- one at a time: from $[n/2]$ to 1
- Inc/Dec key operation: $(\log n)$
- Delete: $(\log n)$ Swap(last, top) then adjust(top).
- Insert: from last then adjust.
- Heap Sort using Heapify.

$$(n + n \log n) = O(n \log n) \text{ delete}$$

- $O(1)$: SC = inplace, not stable.
- max → min heap in just $O(n)$.
- 1st min elem: 1
- 2nd " " : 2
- 3rd " " : 3

Kth Smallest element: $O(K \log n)$

lower Bound: Best case $\Omega(\log n)$

No. of possible Min heap with n distinct no.

$$T(n) = \binom{n-1}{r} \cdot T(r) \cdot T(n-1-r)$$

r = elements in left subtree.

$$T(7) = 80$$

- Smallest elem is always at leaf node in max heap.
- 2nd largest ele is always child of root in max heap.
- max heap can be constructed in $O(n)$.

In insertion comparisons made even in already sorted arr.

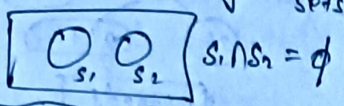
It requires shifting.

$$T(n) = T(n/2) + T(n/2) + n^2$$

$(n^2/4) \dots (4n^2/4) \rightarrow$ combining on tree.

* Set :

- unordered collection of non repeated distinct elements.
- Disjoint Set: Mutually exclusive sets



→ $S_1 - S_2 \neq S_2 - S_1$ not always.

- Root of tree is representative of the set. (usually)

Find(x) = representative of the set of x
 (always > e representative)

Parent(x) = just parent of x

- Kruskal using DSU

can be -1 if no parent

TC: $O(e \cdot \log e)$
 SC: $O(e + v)$

* Test:

- factorial have lack of overlap subproblem.

- Dijkstra prob' & Bellman prob'

- $n^{1.528} \neq O(n^2)$

- $2^n + 2^{n/2} + 2^{n/4} + \dots + 1$ leading term = 2^n

* PYQ:

$$\sum_{i=0}^n \left(\frac{1}{x}\right) = \log n$$

- Advance Master theorem:

$$T(n) = a T\left(\frac{n}{b}\right) + (n^k \log^p n)$$

$$a > b^k ; \Theta(n^{\log_b a})$$

$$a < b^k ; \Theta(n^k \log^p n) \quad p \geq 0$$

$$\Theta(n^k) \quad p < 0$$

$$a = b^k ;$$

$$p > -1 ; \Theta(n^{\log_b a} \log^{p+1} n)$$

$$p = -1 ; \Theta(n^{\log_b a} \log \log n)$$

$$p < -1 ; \Theta(n^{\log_b a})$$

$$\frac{\ln n}{n} \neq \log_2$$

- In an undirected G , Dfs Tree do not have cross edge, always back edge.

an edge can only be tree or back

- T.C of dfs & Bfs on adj Matn

$$O(V^2)$$

- max depth of i th smallest elem' in i th lvl.

